A map respecting $\simeq_S$ is one that descends to $\Gamma/\simeq_S$, and two maps related by $\simeq_S$ are two that descend to the same map there, each respecting it by Lemma 35. Reflexivity is the one property the function former does not preserve: $f \simeq_S f$ demands related outputs at every pair of related inputs and not at the equal ones alone, so it holds of a map exactly where the map descends, which is why respect is a condition rather than a given. Respect at two key sets is two conditions of which neither implies the other, since $\simeq \subseteq \simeq_S$ weakens the hypothesis and the conclusion together. A map that branches on a key outside S is the case that separates them, respecting $\simeq$ and failing to respect $\simeq_S$.

Definition 36. Define the effect function *witnessed up to* $\simeq_S$ as:

$$\begin{aligned}\mathfrak{E}^S_\Gamma := (e : \Gamma \to \Gamma \times (\Gamma \to \Gamma)) \times (e \simeq_S e) \\ \times ((\gamma : \Gamma) \to (\delta : \Gamma) \to (g : \Gamma \to \Gamma) \to ((\delta, g) = e(\gamma) \to g(\delta) \simeq_S \gamma))\end{aligned} \tag{36}$$

The clause $e \simeq_S e$ carries every constituent's respect, its instance at $\gamma \simeq_S \gamma$ relating each yielded inverse to itself. We write $\mathfrak{E}^*_\Gamma$ for $\mathfrak{E}^K_\Gamma$, and taking $\simeq$ to be equality on Γ recovers Definition 8, every map being equal to itself. The key set is where a component's declarations enter: what Section 4 holds an effect function to is $\mathfrak{E}^S_\Gamma$ at the keys that component names (Definition 48).

The iterator of Section 3.1.3 is witnessed the same way, its continuation compared by Definition 34 and witnessed again by the recursion:

Definition 37. Define the effect iterator *witnessed up to* $\simeq_S$ as:

$$\begin{aligned}\mathfrak{I}^S_\Gamma := \mu\mathfrak{I}.\ (e : \Gamma \to \Gamma \times (\Gamma \to \Gamma) \times \mathsf{Maybe}(\mathfrak{I})) \times (e \simeq_S e) \\ \times ((\gamma : \Gamma) \to (\delta : \Gamma) \to (g : \Gamma \to \Gamma) \to (o : \mathsf{Maybe}(\mathfrak{I})) \to ((\delta, g, o) = e(\gamma) \to g(\delta) \simeq_S \gamma))\end{aligned} \tag{37}$$

The embedding of Section 3.1.3 carries $\mathfrak{E}^S_\Gamma$ into $\mathfrak{I}^S_\Gamma$, and taking $\simeq$ to be equality on Γ and $S = K$ recovers the witnessed $\mathfrak{I}^*_\Gamma$ of Definition 17.

Lemma 38. With $\mathfrak{E}^*_\Gamma$ read as in Definition 36, every equality of states asserted in Section 3.1 holds with $=$ replaced by $\simeq$, and the accumulator of every state reachable from $(\gamma_0, \mathrm{id}_\Gamma)$ respects $\simeq$. The same holds of any equivalence substituted for $\simeq$, the proof using no property of the relation beyond transitivity and respect.

Proof. An accumulator is a composition of inverses, each respecting $\simeq$, the clause $e \simeq e$ of Definition 36 read at $\gamma \simeq \gamma$, and a composition of maps respecting $\simeq$ respects $\simeq$, the base case being id_Γ. The proofs of Section 3.1 then go through unchanged, respect being what carries a relation through an inverse: from $g_2(\delta_2) \simeq \delta_1$ and $g_1(\delta_1) \simeq \gamma$ respect gives $(g_1 \circ g_2)(\delta_2) \simeq \gamma$, which is the step each composition of inverses takes, and the soundness invariant of Theorem 7 reads $\varphi(\gamma) \simeq \gamma_0$ by that step. □

The stage shape of Definition 30 settles which readings of $\simeq$ a member admits, and with them its membership in the witnessed iterators, which is what lets a claim about one component be read at the keys that component names.

Lemma 39. Let $i \in \mathfrak{I}^{\mathcal{A}}_\Sigma(S', P)$ and let $S \subseteq K$ contain every key at which a stage of i occurs. Then i lies in $\mathfrak{I}^S_\Sigma$ (Definition 37); in particular i and every inverse it yields respect $\simeq_S$, and the witnesses hold at equality.

Proof. By induction on the construction of Definition 30. The unit yields its argument, id_Σ, and Nothing everywhere. At an operation stage performing $a \in \mathcal{A}_k$, let $\sigma \simeq_S \sigma'$; then $k \in S$, so $\sigma(k) \simeq_k \sigma'(k)$. a respects $\simeq_k$ (Lemma 32), so it is defined at both or at neither and yields $\simeq_k$-related values, equal outcomes, and inverses carrying $\simeq_k$-related values to $\simeq_k$-related values, and its lift reads and writes k alone; the states reached are therefore $\simeq_S$-related, the inverses are $\simeq_S$-related and respect $\simeq_S$, and the equal outcomes select one continuation, to which the induction hypothesis applies. At a provision stage at $k \in P \subseteq S$, the precondition $k \notin \mathrm{dom}(\sigma)$ holds at both or at neither, the states reached bind k to the one value the stage carries, and the restriction it yields respects $\simeq_S$, two related states binding k alike. The witness of each stage holds at equality, the lift of an operation's inverse restoring the value Definition 29 witnesses it to and the restriction reverting the extension (Definition 20), and equality gives $\simeq_S$. □ 1

A stage reads the key it performs at and nothing else, so the hypothesis is met by taking S to be the keys the member names. 2

3.4. Attaining Independence 3

On the strength of Section 3.3, this section supplies the condition that extends the local guarantees to a system of interleaved components. Section 3.4.1 defines the condition, namely *independence* of two effect functions: every transformation of one commutes with every transformation of the other, forward maps and yielded inverses alike. Section 3.4.2 then reduces independence of context-mediated effect functions to commutativity at single coeffects and refines the coeffect with a witness of that commutativity, parallel to the witness of an effect function. 4

3.4.1. Effect Independence 5

Reverting an effect at the state its own application produced is what Theorem 16 covers; reverting one at any other state is what this subsection covers. Two situations call for the latter. An inverse may be run while later effects are still in place, which is what removing one component from a running system amounts to; and one sequence may interleave the effects of several components, each holding the inverses of its own, so that the inverses of one component are separated by the applications of another. In both an inverse meets a state that foreign effects have moved, and whether it still reverts what it was built to revert is a question of commutation: what has to commute is every transformation one effect can perform with every transformation the other can perform, forward map and yielded inverse alike. A single accumulator settles neither situation, φ being a composite that runs every inverse it holds in one order and all at once. 6

An iterator gives rise to several maps: the forward map of each iteration it can reach, and the inverse each yields at each context state. Commutation of two iterators relates the two collections rather than two single maps, and those collections are what the definitions below quantify over. 7

Definition 40. For an iterator $i \in \mathfrak{I}_\Gamma$, let $\mathrm{reach}(i)$ be the least set of iterators containing i and closed under continuation. The *transformation monoid* $\mathfrak{M}(i)$ is the submonoid of $\Gamma \to \Gamma$ generated by the forward maps and the yielded inverses of every iterator in $\mathrm{reach}(i)$, and the *generators* of $\mathfrak{M}(i)$ are the elements of that generating set: 8

$$\mathrm{reach}(i) := \bigcap\{S \mid i \in S \wedge \forall i' \in S, \gamma \in \Gamma.\ i'(\gamma) = (-,-,\mathsf{Just}(i'')) \Rightarrow i'' \in S\} \quad (38)$$
$$\mathfrak{M}(i) := \langle\{\mathrm{pr}_1 \circ i' \mid i' \in \mathrm{reach}(i)\} \cup \{\mathrm{pr}_2(i'(\gamma)) \mid i' \in \mathrm{reach}(i), \gamma \in \Gamma\}\rangle$$

Write $\mathrm{len}(i)$ for the supremum of $|C|$ over the chains $C \subseteq \mathrm{reach}(i)$ that continuation orders. Through the embedding of Section 3.1.3, $\mathfrak{M}(e)$ at an effect function e is generated by the forward map of e together with every inverse e yields; an effect induced by a pair $(f, g) \in \mathfrak{T}_\Gamma$ has f and g for its generators, the inverse it yields being g at every state.

Lemma 41. Commutation is settled on the generators, and $\diamond$ enlarges no transformation monoid. That is,

1. if every generator of $\mathfrak{M}(e_1)$ commutes with every generator of $\mathfrak{M}(e_2)$, then every element of $\mathfrak{M}(e_1)$ commutes with every element of $\mathfrak{M}(e_2)$;
2. $\mathfrak{M}(e_1 \diamond e_2) \subseteq \langle\mathfrak{M}(e_1) \cup \mathfrak{M}(e_2)\rangle$.

Proof.

1. The maps commuting with every generator of $\mathfrak{M}(e_2)$ form a submonoid of $\Gamma \to \Gamma$, since id_Γ lies in it and $f \circ f'$ does where f and f' do. That submonoid contains the generators of $\mathfrak{M}(e_1)$ by hypothesis and hence contains $\mathfrak{M}(e_1)$. Fixing $f \in \mathfrak{M}(e_1)$, the maps commuting with f likewise form a submonoid containing the generators of $\mathfrak{M}(e_2)$ and hence $\mathfrak{M}(e_2)$.
2. By Definition 9 the forward map of $e_1 \diamond e_2$ is $(\mathrm{pr}_1 \circ e_1) \circ (\mathrm{pr}_1 \circ e_2)$ and the inverse it yields at any state is $s \circ t$ for an s yielded by e_2 and a t yielded by e_1. Every generator of $\mathfrak{M}(e_1 \diamond e_2)$ is therefore a composite of generators of the two. □

Definition 42. Iterators $i, j \in \mathfrak{I}_\Gamma$ are *independent* when

1. every transformation of one commutes with every transformation of the other,
$$\forall f \in \mathfrak{M}(i), g \in \mathfrak{M}(j).\quad f \circ g = g \circ f \quad (39)$$
2. neither one's transformations disturb what the other yields, inverse and continuation alike,
$$\forall i' \in \mathrm{reach}(i), g \in \mathfrak{M}(j), \gamma \in \Gamma.\quad \mathrm{pr}_{2,3}(i'(g(\gamma))) = \mathrm{pr}_{2,3}(i'(\gamma)) \quad (40)$$
and the same with i and j exchanged.

A family $(i_l)_{l \in L}$ is *pairwise independent* when i_l and $i_{l'}$ are independent for every $l \neq l'$. A family may repeat an iterator, and holding one independent of itself is holding $\mathfrak{M}(i)$ commutative.

Read at effect functions through the embedding of Section 3.1.3, clause (2) compares the inverse alone, the continuation being Nothing at every state, and the result below is given at effect functions; Section 4 applies the definition at the iterators themselves. For effects induced by pairs (f_1, g_1) and (f_2, g_2), clause (1) is by Lemma 41(1) the commutation of the four pairs f_1, f_2; g_1, g_2; f_1, g_2; and g_1, f_2, and clause (2) holds outright, an induced effect yielding one inverse at every state. Commutation under $\diamond$ is a different property. What $e_1 \diamond e_2 = e_2 \diamond e_1$ equates is the composite forward map of the two orders with each other and the composite inverse of the two orders with each other, each inverse entering the composite at the state its own application produced; independence instead relates each transformation of one effect to each transformation of the other, a forward map paired with a foreign inverse included.

Under independence an inverse may be run at a state later effects have moved, and it withdraws there its own contribution and nothing else, whatever order the inverses are applied in:

Theorem 43. Let $e_1, \cdots, e_n \in \mathfrak{E}^*_\Gamma$ be pairwise independent and applied in order from γ_0, and let each g_i be the inverse e_i yields where it is applied. Applying the n inverses at the state the sequence reaches, in the order of any permutation of $\{1, \cdots, n\}$, reaches γ_0.

Proof. Write $f_i := \mathrm{pr}_1 \circ e_i$, let $\delta_i := f_i(\delta_{i-1})$ with $\delta_0 := \gamma_0$, so that $g_i = \mathrm{pr}_2(e_i(\delta_{i-1}))$. Fix j and write $\delta'_i := (f_i \circ \cdots \circ f_{j+1})(\delta_{j-1})$ for the states of the sequence with e_j omitted, so that $\delta'_j = \delta_{j-1}$. Two claims hold for every u with $j \le u \le n$.

(1) $\delta_u = f_j(\delta'_u)$ and $g_j(\delta_u) = \delta'_u$. The first equation is an induction on u: at $u = j$ it reads $\delta_j = f_j(\delta_{j-1})$, which is the definition of δ_j, and for the inductive step, $\delta_{u+1} = f_{u+1}(\delta_u) = f_{u+1}(f_j(\delta'_u)) = f_j(f_{u+1}(\delta'_u)) = f_j(\delta'_{u+1})$, the middle equality being clause (1) of Definition 42 for e_{u+1} and e_j, which are distinct effects of the family since $u + 1 > j$. For the second equation, clause (1) carries g_j out through the forward maps applied after e_j, leaving the witness of e_j to be used at the one state it holds at:

$$g_j(\delta_u) = (g_j \circ f_u \circ \cdots \circ f_{j+1})(\delta_j) = (f_u \circ \cdots \circ f_{j+1})(g_j(f_j(\delta_{j-1}))) = \delta'_u$$

the last equality resting on $g_j(f_j(\delta_{j-1})) = \delta_{j-1}$, which is the witness Definition 8 requires of e_j at δ_{j-1}.

(2) Each e_i with $i > j$ yields at δ'_{i-1} the same inverse g_i it yields at δ_{i-1}: by (1) the state δ_{i-1} is $f_j(\delta'_{i-1})$, and $f_j \in \mathfrak{M}(e_j)$, so clause (2) of Definition 42 for e_i and e_j gives $\mathrm{pr}_2(e_i(f_j(\delta'_{i-1}))) = \mathrm{pr}_2(e_i(\delta'_{i-1}))$.

The theorem follows by downward induction on n. Let the permutation begin with j. By (1) applying g_j at δ_n reaches δ'_n, the state the sequence with e_j omitted reaches, and by (2) the inverses the remaining effects yielded there are the g_i in hand. That sequence is pairwise independent, being a subfamily, so the induction hypothesis applies to it and to the rest of the permutation; the empty sequence reaches γ_0. □

Section 4.3.2 carries this conclusion to a trace of a whole system, where the steps of other fibers intervene between an effect and its revert.

3.4.2. Coeffect Commutativity

The commutation Definition 42 requires is read up to $\simeq$ by Lemma 38, a yielded continuation compared as Definition 34 compares two iterators, and reading it that way is what makes it attainable at all: two operations may leave values that $\simeq_k$ identifies and still count as commuting. Of two operations it requires one thing more than of the effect functions their lifts induce, an operation yielding an outcome as well.

Definition 44. Operations a and a' are *independent* when their lifts are independent as effect functions (Definition 42) at every pair of arguments, and neither one's transformations disturb the outcome the other yields:

$$\forall x : X_a,\ g \in \mathfrak{M}(a'^{\Sigma}),\ \sigma \in \Sigma.\quad \mathrm{pr}_3(a^{\Sigma}(x)(g(\sigma))) = \mathrm{pr}_3(a^{\Sigma}(x)(\sigma)) \tag{41}$$

and the same with a and a' exchanged, writing $\mathfrak{M}(a^{\Sigma})$ for the submonoid generated by the forward maps and yielded inverses of the lifts $a^{\Sigma}(x)$ over every argument, as Definition 40 generates $\mathfrak{M}$. A key k is *commutative* when any two operations of $\mathcal{A}_k$ are independent, an operation being held independent of itself as well.

Across distinct keys the condition holds outright.

Theorem 45. Operations at distinct keys are independent.

Proof. Let a lie in $\mathcal{A}_k$ and a' in $\mathcal{A}_{k'}$ with $k \neq k'$. By Definition 29 every generator of $\mathfrak{M}(a^{\Sigma})$ is of the form $\sigma \mapsto \sigma[k \mapsto u(\sigma(k))]$ for a map u on $\mathcal{V}_k$, being either the lift of a forward map or the lift of a yielded inverse, and likewise for a' at k'. Two such maps commute, each reading and writing one key alone and the two keys differing, and Lemma 41(1) extends the commutation from the generators to the two monoids. For the second condition, what a^{Σ} yields at σ, inverse and outcome alike, is determined by $\sigma(k)$, which every generator of $\mathfrak{M}(a'^{\Sigma})$ leaves as it stands. □

The condition therefore turns on the pairs at one key, and the proof of their independence is made a constituent of the coeffect itself, as the proof that an inverse reverts is a constituent of the effect function (Definition 8):

Definition 46. A coeffect at k (Definition 29) is *witnessed* when it carries, as a third constituent beside $\mathcal{V}_k$ and $\mathcal{A}_k$, a proof that k is commutative (Definition 44).

The two witnesses are parallel: each certifies the condition its consumers would otherwise have to assume, the returned inverse reverting there and the operations commuting here, and each is supplied where the definition is written rather than checked where it is used. By Theorem 45 the proof concerns the operations of k alone, so the obligation falls on the component providing the key and on no component consuming it; the examples below are how it is discharged. From here on every coeffect is witnessed, and Section 4 reads every key of K so.

A key whose value is a table of entries is commutative when each registration takes an entry of its own, registration of a route or of an event listener being the representative case. The operation draws an identifier for the entry it adds and the inverse it yields removes that entry, so two registrations name two entries whatever they register: either order leaves a table that answers every test alike, and either registration can be withdrawn while the other stands. Replicated data types are designed to this condition and attach a unique tag to each addition for this very reason, a set whose additions and removals name a bare element having no such property [43]. A key whose value is an ordered chain is not commutative, since a middleware inserted before another sees a different request, and neither order can be withdrawn without disturbing the other.

The allocator of the opening example divides by what its interface publishes. Where the handles it hands out are compared by no operation of the key, no test observes them, so $\simeq_k$ relates two heaps up to a renaming of handles and allocation is commutative; a renaming is an equivalence the operations respect, contained in $\simeq_k$ by Lemma 32(2), and it is how CompCert relates the memory states of a program and of its translation [44]. Where the addresses are outcomes compared by equality, the outcome of a further allocation separates the two orders of allocation, and the key is not commutative. POSIX draws the same line across its own allocators: `mmap` may return any unused address and `creat` may assign any unused inode, whereas `open` is required to return the lowest available descriptor, and that requirement alone is what stops two descriptor allocations from commuting [45].

Definition 31 turns each of these divisions into a design choice. $\simeq_k$ is indistinguishability under the tests the operations of k generate, so an interface publishing fewer outcomes admits fewer tests and coarsens the relation, and withholding an outcome its callers do not need can carry a key from one side of a division to the other. The scalable commutativity rule applies

the same move across the POSIX interface, and reads commutativity as indistinguishability through an interface rather than equality of internal states [45].

Independence of two context-mediated iterators (Definition 30) turns on their keys alone:

Theorem 47. Let $i_1 \in \mathfrak{I}^{\mathcal{A}}_{\Sigma}(S_1, P_1)$ and $i_2 \in \mathfrak{I}^{\mathcal{A}}_{\Sigma}(S_2, P_2)$ with $P_1 \cap S_2 = P_2 \cap S_1 = \varnothing$, and let every key at which operations of both occur be commutative (Definition 44). Then i_1 and i_2 are independent (Definition 42).

Proof. Every iterator $\mathrm{reach}(i_l)$ contains is the unit or a stage, whose forward map and yielded inverses are the constituents of the operation or the set its head performs, so the generators of $\mathfrak{M}(i_l)$ are among those of the stages occurring in i_l, together with id_{Σ}.

For clause (1) of Definition 42 it is enough, by Lemma 41(1), that those generators commute pairwise. Each is *key-local*: defined by the binding at one key alone, presence included, and writing that binding alone, being the lift of an operation's forward map or of an inverse it yields (Definition 29), the extension a provision stage takes, or the restriction it yields (Definition 20). Two key-local maps at distinct keys commute, each leaving what the other reads and writes as it stands. This settles every pair involving a provision-stage generator, whose key lies in one P_l and hence outside the other member's every key by hypothesis, and every pair of operation generators at distinct keys, which is Theorem 45; a pair of operation generators at one key is covered by that key's commutativity.

For clause (2), take $i' \in \mathrm{reach}(i_1)$, $g \in \mathfrak{M}(i_2)$, and $\sigma \in \Sigma$. The unit yields $(\mathrm{id}_{\Sigma}, \mathsf{Nothing})$ everywhere. A provision stage at $k \in P_1$ yields the restriction at k and its one continuation whatever the state, and is defined at σ and $g(\sigma)$ alike, its precondition reading the presence of k, which every generator of $\mathfrak{M}(i_2)$ leaves as it stands. An operation stage at $k \in S_1$ yields what $a^{\Sigma}(x)$ yields at $\sigma(k)$, inverse and outcome; where no operation of i_2 occurs at k the generators of $\mathfrak{M}(i_2)$ leave $\sigma(k)$ as it stands, and where one does the key is commutative by hypothesis, and independence of its operations, applied to one generator of g at a time, yields the same inverse and the same outcome at $g(\sigma)$. Equal outcomes select one continuation, so the yields agree. □

With every coeffect witnessed, the commutativity hypothesis of Theorem 47 is supplied at every key (Definition 46), and only the disjointness $P_1 \cap S_2 = P_2 \cap S_1 = \varnothing$ remains to be checked of a pair; Section 4 reads that disjointness off the two components' declarations.

A component's effect function is the lift of a context-mediated iterator along the coeffect projection (Definition 56), and independence transfers to that lift, whose transformations move the projection alone. The assumption Section 3.4.1 leaves open is met that way, the witness of each coeffect supplying the commutativity Theorem 47 consumes, and with it the temporal composability of a whole system of components.

What the decomposition divides is a computation's commuting part from its order-sensitive part. The commuting part is carried by the effects: a component performs them in whatever order its task calls for, and Theorem 43 reverts them in whatever order the system finds convenient, no two components constraining each other. The order-sensitive part is carried by the coeffects, since a key whose operations do not commute is one whose order has to be imposed from outside the effects, and two places are available for imposing it. Within one component the accumulator imposes it, reverting in LIFO order whatever the effects (Theorem 16). Across components a declared coeffect imposes it, one component providing what another declares and the provision preceding the declaration's satisfaction (Section 3.2.2). Composability is

thereby had at the grain of components rather than of single effects, which is the scale Section 4 works at.

One limit of the theorem is worth naming, and it is the hypothesis this section opened on: binding every shared location at a key is the paradigm's discipline and not a property of the construction, so a location the system cannot reify as a coeffect lies outside the boundary of Section 6.1 and outside the theorem with it.

4. A Calculus of Dynamic Composition

This section gives the theory of Section 3 an operational semantics. It decomposes a running system into *components*, each a triple of a coeffect specification, a provision, and a witnessed effect function. The instantiations of components are *fibers*, and the calculus supplies the rules that move them: orchestration rules, by which the orchestrator inserts and retires fibers, and lifecycle rules, by which the system activates and deactivates them unprompted. The metatheory then establishes temporal and spatial composability in their global form, the guarantees Section 3 reads of one component holding of every fiber of an arbitrary interleaving.

4.1. Components and Fibers

This section fixes the objects the rules act on: the *component*; the *fiber*, an instantiation of a component carrying a lifecycle state of its own; and the *registry*, which holds the fibers a state carries and from which the coeffect context is read off.

Components. A component is given as a triple, its coeffect side split into what it reads from the environment and what it provides to it.

Definition 48. A *component* over a context Γ carrying both effects and coeffects (Definition 28) is defined as:

$$\mathfrak{C}_\Gamma := (d : \mathfrak{D}_\Gamma) \times (p : \mathfrak{P}_\Gamma) \times \mathfrak{I}_\Gamma^{d \cup p} \tag{42}$$

representing a triple (d, p, e), where:

- $d : \mathfrak{D}_\Gamma$ is the coeffect specification of Definition 21, declaring the dependencies required from the environment;
- $p : \mathfrak{P}_\Gamma := \mathsf{Set}(K)$ is the *coeffect provision*, declaring the coeffect keys the component may provide, and no key outside p is one its effect function installs a binding at;
- $e : \mathfrak{I}_\Gamma^{d \cup p}$ is the witnessed effect function, an effect iterator (Definition 17) witnessed up to $\simeq_{d \cup p}$ (Definition 37), defining the effects contributed when the component is active together with the inverses that withdraw them; a plain effect function enters through the embedding of Section 3.1.3.

Subscripts are taken on Γ throughout, the coeffect context being one of its projections (Definition 28), so the $\mathfrak{D}_\Sigma$ of Definition 21 is written $\mathfrak{D}_\Gamma$ here.

Fibers. One component may be instantiated many times over, and each instantiation is activated and deactivated over time, carrying a lifecycle state of its own. We name such an instantiation a *fiber*. A fiber records the component that produced it, the fiber it was instantiated under, the coeffects it provides, and where in its lifecycle it stands.

Definition 49. Fix a set $\mathfrak{N}$ of fiber names. A *fiber* instantiating the component $(d, p, e) \in \mathfrak{C}_\Gamma$ is a tuple $\langle d, p, e, \pi, \sigma, \tau, \theta \rangle$, where

- $d : \mathfrak{D}_\Gamma$, $p : \mathfrak{P}_\Gamma$, and $e : \mathfrak{I}_\Gamma^{d \cup p}$ are the coeffect specification, provision, and effect function of Definition 48;
- $\pi : \mathfrak{N} \cup \{\mathsf{root}\}$ is the *parent*, the fiber this one was instantiated under, or the root marker root;
- $\sigma : \Sigma$ is the fiber's own coeffect table (Definition 19), empty until it activates and written by its effects as they run;
- $\tau : \{\bot, \top\}$ is the *retirement* flag, $\bot$ in a fresh fiber and $\top$ once the orchestrator has retired the fiber;
- $\theta : \Theta_\Gamma$ is the *lifecycle state*:

$$\Theta_\Gamma := \mathsf{Inactive} \mid \mathsf{Reloading}(i, g, \omega) \mid \mathsf{Active}(g, \omega) \mid \mathsf{Unloading}(g, \omega) \tag{43}$$

  where $i : \mathfrak{I}_\Gamma^{d \cup p}$ is the remaining effect iterator, $g : \Gamma \to \Gamma$ the accumulator built so far, and $\omega : d \to \mathfrak{N}$ the *committed view*.

A fiber is *installed* when its lifecycle state carries an accumulator and a committed view:

$$\mathrm{installed}_n(\gamma) := \theta_n \neq \mathsf{Inactive} \tag{44}$$

and an installed fiber *resolves k to m* when $\omega_n(k) = m$.

A *transition* is what moves a fiber from one lifecycle state to another, and between transitions the fiber rests at one of the two *settled* states, Inactive or Active, contributing nothing at the first and its effects at the second. A transition runs in one of two directions: an *activation* executes e, accumulating side effects on the context, and a *deactivation* applies the accumulator to recover the context. A transition in a real runtime is spread over an interval rather than taken in one step, so each direction has a state of its own that the fiber occupies while the transition runs, Reloading for an activation and Unloading for a deactivation. The committed view ω sends each key the fiber declares to the name of the fiber that provided it when the transition committed. Section 4.2 draws the four states as a state machine (Figure 1) and supplies the rules on its edges.

Registry. A state holds its fibers under their names, and both the identity of a fiber and the coeffect context of Section 3.2 are read off that arrangement.

Definition 50. Write $\mathfrak{F}_\Gamma$ for the set of fibers over Γ. A state $\gamma \in \Gamma$ carries a *registry*

$$F_\gamma : \mathfrak{N} \rightharpoonup \mathfrak{F}_\Gamma \tag{45}$$

a finite partial function whose parent pointers form a tree rooted at root, together with whatever else in Γ no fiber's σ names. We write $\gamma(n)$ for $F_\gamma(n)$, and abbreviate a field of $\gamma(n)$ by subscripting it with n where the state is clear, so that $d_n, p_n, e_n, \pi_n, \sigma_n, \tau_n, \theta_n$ are the fields of Definition 49 and g_n, ω_n the accumulator and committed view that θ_n carries; $\gamma[\theta_n \mapsto \theta']$, $\gamma[n \mapsto \langle \cdots \rangle]$, and $\gamma \setminus n$ are the states differing from γ in one field, one fiber, and the presence of one fiber respectively.

A fiber's name is what gives it an identity that survives its own mutation: every rule below rewrites the lifecycle state of one fiber and leaves the others alone, so the rule has to say which one, and two fields refer to fibers rather than describe them, the parent π and the committed view ω. Names are atoms: no rule computes one, inspects its structure, or relates two of them by anything but equality, and introducing a fiber simply draws one not already in use. This is the discipline of dynamically created local names [40], used here for fiber identity.

Each fiber owning a table means the coeffect context is derived rather than stored: it is what the active fibers jointly provide. 1

$$\sigma_\gamma := \bigcup \{ \sigma_m \mid m \in \mathrm{dom}(F_\gamma),\, \theta_m = \mathsf{Active}(-,-) \} \quad (46)$$ 2

The union is well defined because a fiber's own table holds only the keys of its provision, $\mathrm{dom}(\sigma_n) \subseteq p_n$ (Definition 48), and the provisions of distinct fibers are disjoint, O-Insert admitting no fiber whose provision meets an existing one (Section 4.2.1), so each $k \in \mathrm{dom}(\sigma_\gamma)$ lies in the table of exactly one Active fiber, whose name we write $\mathrm{provider}_k(\gamma) \in \mathfrak{N}$ and call the *provider* of k. Each key therefore has one possible provider, fixed by the provisions and not by the state. No rule writes a table directly: the bindings a fiber provides are the provision stages its own effect function performs, which land in σ_n and so are already part of the state e_n returns, and they leave again with the accumulator, and the operations it performs at a declared key act on the value its provider's table holds (Definition 56). An effect function is built from such stages and from nothing else, so a location no key binds is one no fiber touches. 3

The disjointness the union rests on is where this chapter parts company with Section 3.2.3, and it simplifies the formalization rather than the systems it models. The isolation of Definition 24 lets one key resolve through a realm table, so that two fibers may provide the same key in different realms; a calculus carrying realms would relax disjointness to disjointness within a realm, resolving a declared key against the realm of the fiber declaring it (Section 4.4 supplies that reading). We read every key at one shared realm instead, and a system that wants several providers of one key keeps two routes: realms, and the broker of Section 6.2, one fiber providing the key and dispatching among implementations registered with it. Within the calculus, the disjointness restricts how often a component may be instantiated: one with a nonempty provision has one fiber at a time, so the many instantiations below are of components providing nothing, which is the common case of a component that only consumes, or that instantiates others. 4

With σ_γ in hand, the satisfaction relation of Section 3.2.2 applies unchanged, $\gamma \vDash d$ abbreviating $\sigma_\gamma \vDash d$. A key lies in $\mathrm{dom}(\sigma_\gamma)$ exactly when some Active fiber has installed it, its provision being the keys it may install rather than the ones it has, so $\gamma \vDash d$ already requires that every declared key have an Active provider. Taking the union over Active fibers alone is what lets a fiber cease to provide before it has withdrawn anything, which Section 4.2.2 turns into the ordering discipline, and it fixes how a transition in progress reads: a Reloading or Unloading fiber reads its coeffects through the ω it holds and provides none of its own, so a key its transition has already written is not yet one a dependent may activate against. 5

The two declarations of Definition 48 are the two directions of one *interface*, d what a component reads from the environment and p what it writes to it, and the superscript on the effect function's type holds the witness to that interface. A fiber holds its own bindings whether or not they are published, so the projection the witness is read at is a second reading of the same tables. 6

Definition 51. Write 7

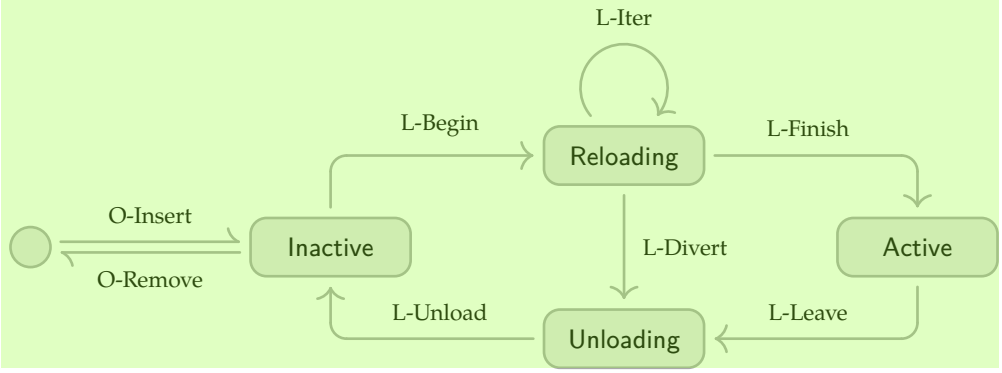
$$\sigma_\gamma^S := \bigcup \{ \sigma_m|_S \mid m \in \mathrm{dom}(F_\gamma) \} \quad (47)$$ 8

for the bindings the registry records at a set $S \subseteq K$ of keys, over every fiber and not the Active ones alone; disjointness of provisions makes it a function, and σ_γ is the restriction of σ_γ^K to the Active fibers. This is the coeffect projection Definition 33 is read at throughout this section, so that $\gamma \simeq_S \delta$ is $\sigma_\gamma^S \simeq \sigma_\delta^S$ and $\mathfrak{I}_\Gamma^S$ (Definition 37) is the effect functions witnessed at those keys. 9

The keys of both declarations are read off the tables rather than off σ_γ, and the witness condition is why: a binding a transition has written stays in the fiber's table before the fiber is Active, and that binding is what the inverse is held to remove, so the projection the witness is read at has to hold it wherever the fiber's lifecycle stands. The same reading keeps the relation where the control fields cannot move it: a write to a lifecycle state can move a table into or out of σ_γ with every binding left as it stands, whereas σ_γ^S moves only where some binding does. What the witness is thereby held to restore is the two declarations and nothing else. 1

4.2. The Calculus 2

This section gives the calculus: nine rules generating two relations. An *orchestration* rule, prefixed O- and written $\gamma \Rightarrow \delta$, is an action the orchestrator may perform; its premises say when the action is legal, not when it occurs. A *lifecycle* rule, prefixed L- and written $\gamma \longrightarrow \delta$, is a step the system takes unprompted whenever its premises hold. A sequence of steps interleaves the two. Eight of the nine lie on the edges of Figure 1; O-Retire writes the retirement flag alone and applies at every lifecycle state, so it would be a self-loop at each of the four nodes, and the figure omits it. 3



4

Figure 1 | The component lifecycle, with the empty node marking a fiber absent from the registry 5

4.2.1. *Orchestration* 6

Insertion and retirement are the only external inputs: the orchestrator requests that a fiber exist or stop existing, and never sets its lifecycle state directly. 7

$$\frac{n \notin \mathrm{dom}(F_\gamma) \quad \pi \in \mathrm{dom}(F_\gamma) \cup \{\mathsf{root}\} \quad (d,p,e) \in \mathfrak{C}_\Gamma \quad \forall m \in \mathrm{dom}(F_\gamma).\ p \cap p_m = \varnothing}{\gamma \Rightarrow \gamma[n \mapsto \langle d,p,e,\pi,\varnothing,\bot,\mathsf{Inactive}\rangle]}\ \text{O-Insert}$$

$$\frac{n \in \mathrm{dom}(F_\gamma)}{\gamma \Rightarrow \gamma[\tau_n \mapsto \top]}\ \text{O-Retire}$$

$$\frac{\tau_n = \top \quad \theta_n = \mathsf{Inactive} \quad \sigma_n = \varnothing \quad \forall m.\ \pi_m \neq n}{\gamma \Rightarrow \gamma \setminus n}\ \text{O-Remove}$$

8

O-Retire is unconditional on the fiber's state because retiring is a request, and the lifecycle rules are what carry it out. Retirement is separated from removal for the same reason: a retired fiber that is still Active must first be deactivated, and removing it earlier would discard the accumulator and leak. The premise $\forall m.\ \pi_m \neq n$ keeps the tree well-formed by removing children before their parent, and $\sigma_n = \varnothing$ admits only an entry holding no bindings, so that a removal discards none: a deactivation leaves the entry so (Corollary 69), and it is what lets Theorem 68 count a removal as no change to the tables. The last premise of O-Insert is where the 9

single-source discipline is imposed: a key has one possible provider because the orchestrator may not admit a second component declaring it.

Instantiation. A component may instantiate another while installing its effects, which is what a plugin host does when a plugin loads plugins of its own. The rules so far leave the registry to the orchestration rules alone, so such an instantiation has nowhere to happen. One primitive gives it somewhere.

Definition 52. An iteration of e_n may *instantiate* a component $(d, p, e) \in \mathfrak{C}_\Gamma$. In place of a state map it takes the O-Insert of that component with $\pi = n$, and it yields as its inverse the O-Retire of the fiber so instantiated. The rule draws the name, subject to the freshness premise of O-Insert, and hands it to the effect function.

The inverse retires rather than removes, and the reason is that an inverse has to apply wherever it is reached. O-Remove carries premises, so an inverse built from it can fail to: a parent whose child is still Active could not run its accumulator, and no rule would move the child, since Definition 53 does not read the fiber tree. O-Retire has $n \in \mathrm{dom}(F_\gamma)$ as its only premise. The entry it leaves behind at the state the instantiation was taken is retired, Inactive, and holds an empty table, which is the vestigial entry of Lemma 62: it differs from the absence of the fiber in control fields alone, and no rule tells the two apart.

Retiring a child sets τ and so takes its target view to $\bot$, after which the ordinary rules carry it back to Inactive. The parent is not made to wait, O-Retire being unconditional, so L-Unload applies to the parent whether or not the child has left. A grandchild is reached one level at a time, the child's own accumulator retiring what the child instantiated. Theorem 73 covers this cascade and the one the guard of Section 4.2.2 imposes along coeffects together.

4.2.2. Lifecycle

The six lifecycle rules divide by the direction they move a fiber in: an activation carries a fiber toward a target view it does not yet hold, and a deactivation carries one away from a committed view that is no longer its target.

Target views. The rules compare each fiber against a *target*, namely whether it ought to be running and against which resolution of its dependencies. The target is not a property of the fiber alone, since the keys a fiber declares are resolved against the whole state, so it is a predicate on that state.

Definition 53. The *target view* of n at γ maps each declared key to its provider, so it is a total map $d_n \to \mathfrak{N}$, and is $\bot$ when n ought not to be running at all:

$$\mathrm{target}_n(\gamma) := \begin{cases} \bot & \text{if } \tau_n \vee \neg(\gamma \vDash d_n) \\ (k \in d_n) \mapsto \mathrm{provider}_k(\gamma) & \text{otherwise} \end{cases} \tag{48}$$

A state is *quiescent* when every fiber has settled at its target view, no transition left in progress:

$$\mathrm{quiet}(\gamma) := \forall n \in \mathrm{dom}(F_\gamma).\ \begin{cases} \mathrm{target}_n(\gamma) = \bot & \text{if } \theta_n = \mathsf{Inactive} \\ \mathrm{target}_n(\gamma) = \omega_n & \text{if } \theta_n = \mathsf{Active}(-, \omega_n) \\ \bot & \text{otherwise} \end{cases} \tag{49}$$

The target answers to two things and to nothing else: retirement, through τ_n, and coeffect resolution, through $\gamma \vDash d_n$ and $\mathrm{provider}_k$, each declared key being read off σ_γ at the one shared realm of Section 4.1.

The committed view of Definition 49 has the same type as the target view, and the lifecycle is driven by comparing them: ω_n is the resolution n activated against, $\mathrm{target}_n(\gamma)$ the one it should be running against, and every rule below fires on their agreeing or differing. This is the reactive discipline of Section 3.2: a transition is initiated whenever the target view changes, regardless of which of the two moved it. Recording a provider rather than a value is what makes the comparison usable, since a different fiber providing an equal value would otherwise compare equal. The value a component reads is reached through the view, since the provider's table holds that value, and the implementation holds the map in `fiber.committed` and a hash of it in `fiber.target` (Section 5.1.3).

Activation. An activation may execute multiple effects in sequence, and the deactivation must revert them. The effect iterator e_n models such an activation (Section 3.1.3), each of its iterations yielding the modified context, an inverse, and a continuation, and a component's whole activation is one run of e_n: L-Begin enters Reloading, L-Iter takes one iteration, and L-Finish lands the last.

$$\frac{\theta_n = \mathsf{Inactive} \quad \omega = \mathrm{target}_n(\gamma) \neq \bot}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathsf{Reloading}(e_n, \mathrm{id}_\Gamma, \omega)]}\ \text{L-Begin}$$

$$\frac{\theta_n = \mathsf{Reloading}(i, g, \omega) \quad \mathrm{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \mathsf{Just}(i'))}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Reloading}(i', g \circ h, \omega)]}\ \text{L-Iter}$$

$$\frac{\theta_n = \mathsf{Reloading}(i, g, \omega) \quad \mathrm{target}_n(\gamma) = \omega \quad i(\gamma) = (\delta, h, \mathsf{Nothing})}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Active}(g \circ h, \omega)]}\ \text{L-Finish}$$

Each iteration composes the newly yielded inverse onto the accumulator as $g \circ h$, following Definition 18, so that the accumulator applies the inverses in last-in-first-out order.

Deactivation. A deactivation enters from either stage of the lifecycle: L-Divert takes out a fiber whose transition is still in progress, and L-Leave one that is Active. It cannot be taken in one step. A component being torn down because its provider is going away is running its own teardown code, which may need the very coeffect that is being withdrawn; closing a connection pool typically means handing the connections back to whatever provided them. The consumer must therefore still be able to read the key throughout its own deactivation, and the provider's withdrawal must take effect only afterwards, which gives content to the ordering Section 3.2 requires of dependencies and dependents. A deactivation taken in one step would remove the provisions and run the inverse together, with no interval between them for a consumer's teardown to occupy. The rules below therefore separate the decision from the act, and the act is guarded by the following condition.

Definition 54. The fiber n is *relied upon* at γ when some other installed fiber resolves a key to it:

$$\mathrm{relied}_n(\gamma) := \exists m \in \mathrm{dom}(F_\gamma),\, k \in d_m.\ m \neq n \wedge \mathrm{installed}_m(\gamma) \wedge \omega_m(k) = n \qquad (50)$$

$$\frac{\theta_n = \mathsf{Reloading}(i, g, \omega) \quad \mathrm{target}_n(\gamma) \neq \omega \quad (\delta, h) = (\gamma, \mathrm{id}_\Gamma) \vee i(\gamma) = (\delta, h, -)}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Unloading}(g \circ h, \omega)]}\ \text{L-Divert} \quad 1$$

$$\frac{\theta_n = \mathsf{Active}(g, \omega) \quad \mathrm{target}_n(\gamma) \neq \omega}{\gamma \longrightarrow \gamma[\theta_n \mapsto \mathsf{Unloading}(g, \omega)]}\ \text{L-Leave}$$

$$\frac{\theta_n = \mathsf{Unloading}(g, \omega) \quad \neg\, \mathrm{relied}_n(\gamma) \quad g(\gamma) = \delta}{\gamma \longrightarrow \delta[\theta_n \mapsto \mathsf{Inactive}]}\ \text{L-Unload}$$

L-Divert may fall between any two consecutive iterations of a transition, routing the fiber into Unloading with the inverses accumulated so far rather than applying them on the spot. Routing through Active instead would let the fiber provide its coeffects for the length of one step and oblige its dependents to activate against a component that is already leaving. The first of L-Divert's two alternatives *aborts* the iteration the fiber is holding, which only an iteration boundary makes possible, so the granularity at which a divert may fall is that of the iterator; the second lets that iteration land, serving the host Section 4.4 admits, in which an iteration in flight cannot be declined. 2

L-Leave records the decision to deactivate without acting on it, which stops the fiber providing its coeffects while leaving its own committed view and everyone else's intact. L-Unload applies the accumulator, discards the committed view, and leaves the fiber Inactive; it is the only rule in the calculus that applies an accumulator, an L-Divert routing here rather than applying one of its own. 3

The two requirements are then carried by different parts of the form: the consumer's reading by the committed view, which L-Unload discards as its last act, and the deferral of the withdrawal by the premise $\neg\, \mathrm{relied}_n(\gamma)$, which we call the *guard* and which holds a provider's withdrawal back until every consumer that resolves a key to n has gone. For a fiber L-Divert takes out of its first transition the guard is vacuous, a fiber that has never been Active providing nothing and appearing in no committed view. Theorem 70 establishes both requirements. 4

The guard is imposed per binding rather than per fiber: $\mathrm{relied}_n(\gamma)$ tests whether some committed view names n, so a fiber that declares none of n's keys is no obstacle, and neither is one that resolved a key of n's in another realm (Section 3.2.3). Under the single-source discipline of O-Insert the per-binding reading coincides with the coarser test $\exists m \neq n,\, k \in d_m.\ \mathrm{installed}_m(\gamma) \wedge k \in p_n$, a key having one possible provider there. 5

A guard of this kind ordinarily deadlocks. What keeps it from doing so is Unloading together with σ_γ being the union over Active fibers alone: once L-Leave or L-Divert has marked n, its table leaves σ_γ, so no target view can name n any longer, and every consumer that committed to n is itself on its way out. Theorem 73 turns that into the claim that the guard always releases. 6

The guard orders deactivations along coeffects and not along the fiber tree: a parent may run its inverse while a child of it is still Unloading, since relied speaks only of committed views. Parent and child are accordingly ordered more weakly than Theorem 70 orders a provider and its consumer, and a parent and a child whose effects meet at a shared key are governed by the pairwise independence of Lemma 66 instead. 7

The rules are nondeterministic: several fibers may hold a committed view differing from their target view, and the relation commits to no order among them. They are also *reactive only*, in that no rule mentions a scheduler; the steps are any sequence of rule applications, so a theorem proved over all such sequences holds for every scheduling policy a runtime might adopt. 8